

Team Delivery Package: Preprocessing Engine

Enterprise Milestone-Based Technical Specifications

MODULE NAME:	Team 4: Preprocessing Engine
GIT BRANCH:	feature/preprocessing-features
LEAD MEMBERS:	Basma Mansour, Bothaina Elqady
WORKLOAD TIME:	3 weeks
KICKOFF DATE:	11 July 2026
CODE FREEZE:	23 July 2026
VERSION:	v1.2.0

1. Cover Page & Metadata Summary

This delivery package establishes the contract specifications for the **Team 4: Preprocessing Engine**.

Milestone	Target Date	Status	Description
Project Kickoff	11 July 2026	COMPLETED	Milestone scope locked & repository structured.
Code Freeze	23 July 2026	IN PROGRESS	All functional packages and tests locked.
Integration Phase	24 July 2026	PLANNED	Multi-team pipeline joins and bug resolution.
Final Presentation	25 July 2026	PLANNED	Final delivery and platform presentation.

2. Team Overview & Assignments

Resource Details

Lead Team Members: **Basma Mansour, Bothaina Elqady**
Branch Space: ``feature/preprocessing-features``
Role Assignment: **Data Processing & Feature Engineers**
Difficulty Rating: **High** | Priority Index: **Critical** | Est. Workload: **3 weeks**

3. Business & Technical Goals

Business Objective

Handle anomalies, null cells, and prepare categorical features for predictive modeling.

Technical Objective

Build robust column imputers, one-hot encoders, and datetime dimension extractors.

Position inside AutoAnalyst AI

Data cleaning and transformation layer.

4. System Architecture & folder layout

Assigned folder Tree Structure

```
src/autoanalyst/  
  preprocessing/  
  ├── __init__.py  
  ├── cleaner.py  
  └── feature_engineering/  
      ├── __init__.py  
      └── feature_builder.py
```

Target Code Modules

Your codebase modifications belong inside: `src/autoanalyst/preprocessing/cleaner.py`,
`src/autoanalyst/feature\_engineering/feature\_builder.py`.

Functional Code Interface Specifications

```
# src/autoanalyst/preprocessing/cleaner.py
import pandas as pd

def remove_duplicates(df: pd.DataFrame) -> pd.DataFrame:
    pass

def handle_missing_values(df: pd.DataFrame, strategy: str = "median") -> pd.DataFrame:
    """Imputes missing cells. Strategies: mean, median, mode, drop."""
    pass
```

API Specification

```
handle_missing_values(df: pd.DataFrame, strategy: str) -> pd.DataFrame
Input Configs:
- strategy: 'mean', 'median', 'mode', or 'drop'
Output:
- Clean DataFrame with zero NaN entries in selected columns.
```

5. Git Workflow & Code Integration Policy

Branching Workflow

All developers pull and write inside `feature/preprocessing-features` created from the latest `develop` branch.

Pull Request Requirements

1. Merge latest develop branch changes into features before raising PRs.
2. Assign team lead reviews and resolve conflicts.
3. A minimum of one approved review check is required.

6. Quality Standards & Guidelines

Format & Lint Rules

All variables, methods, and file layouts must use lower_case snake_case parameters. Code comments must be in English.

Error Checking & Imputers

All functional blocks should be guarded by custom exception checks (raise ValueError/KeyError).

Logging Rules

Incorporate logging warnings rather than std print statements.

7. Expected Deliverables & Verification

Required Files list

`src/autoanalyst/preprocessing/cleaner.py, src/autoanalyst/feature\_engineering/feature\_builder.py`

Verification Code Assertions

Unit test validations are situated in: `tests/test\_preprocessing.py, tests/test\_feature\_engineering.py`.

Pipeline integration assertions belong in: `tests/test\_pipeline.py`.

IMPORTANT GUIDELINE

All functional checks must compile successfully with zero syntax warnings before code freeze.

8. Definition of Done Checklist

- [] All functions compile cleanly with PEP 8 checks.
- [] Unit tests cover all key methods and functions.
- [] Try-except checks isolate external dataframe issues safely.
- [] The PR has been reviewed and verified by Team 1.

9. Common Mistakes & Troubleshooting

Pitfalls to Avoid

- Causing Target Leakage by performing categorical encoding or imputation on targets.
- Crashing when datetime strings cannot be parsed due to wrong date formats.
- One-hot encoding columns with extremely high cardinality, causing memory errors.